

Poisson maximum-likelihood fitting

Why photon histograms use a Poisson model

A time-correlated single-photon counting (TCSPC) histogram contains an integer photon count in each time bin. Let

- y_i be the observed count in bin i ;
- $\mu_i(\boldsymbol{\theta})$ be the count predicted by the decay model;
- $\boldsymbol{\theta}$ contain the fitted amplitudes, lifetimes, time offset, IRF width, and background as applicable.

When photon detections are independent, the count in each bin follows a Poisson distribution:

$$y_i \sim \text{Poisson}(\mu_i(\boldsymbol{\theta})) .$$

The prediction μ_i is in expected photon counts, not a normalized probability. It must be nonnegative and should include every modeled source of counts, including background.

Derivation of the likelihood

The probability of observing y_i photons when the predicted mean is μ_i is

$$P(y_i \mid \mu_i) = \frac{\mu_i^{y_i} e^{-\mu_i}}{y_i!} .$$

Assuming that the histogram bins are independent, the likelihood of the complete histogram is the product of the probabilities for all fitted bins:

$$L(\boldsymbol{\theta}) = \prod_i \frac{\mu_i(\boldsymbol{\theta})^{y_i} e^{-\mu_i(\boldsymbol{\theta})}}{y_i!} .$$

Taking the logarithm converts this product into a sum:

$$\log L(\boldsymbol{\theta}) = \sum_i [y_i \log \mu_i(\boldsymbol{\theta}) - \mu_i(\boldsymbol{\theta}) - \log(y_i!)] .$$

The term $\log(y_i!)$ depends only on the observed data, so it cannot change the fitted parameters. Maximizing the likelihood is therefore equivalent to minimizing the parameter-dependent negative log-likelihood

$$-\log L(\boldsymbol{\theta}) = \sum_i [\mu_i(\boldsymbol{\theta}) - y_i \log \mu_i(\boldsymbol{\theta})] + \text{constant} .$$

Poisson deviance used by FLIMage

FLIMage evaluates the equivalent Poisson deviance, also called the Cash deviance:

$$D(\boldsymbol{\theta}) = 2 \sum_i \left[\mu_i - y_i + y_i \log \left(\frac{y_i}{\mu_i} \right) \right].$$

This is twice the log-likelihood difference between the fitted model and a saturated model that predicts every observed bin exactly. All additional terms depend only on the data, so

$$\arg \min_{\boldsymbol{\theta}} D(\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} L(\boldsymbol{\theta}).$$

For a zero-count bin, $y_i \log(y_i/\mu_i)$ is defined by its limit as zero. Its deviance contribution is consequently

$$D_i = 2\mu_i.$$

Zero-count bins are therefore valid observations and should not be removed merely because their observed count is zero. FLIMage floors a predicted value at 10^{-12} during the likelihood calculation to keep logarithms and divisions finite.

FLIMage stores a signed deviance residual for each bin:

$$r_i = \text{sign}(y_i - \mu_i) \sqrt{2 \left[\mu_i - y_i + y_i \log \left(\frac{y_i}{\mu_i} \right) \right]}.$$

Thus, $\sum_i r_i^2 = D$. The reported `xi_square` field is $D/(N - P)$ in Poisson ML mode, where N is the number of fitted bins and P is the number of free parameters. It is a reduced-deviance diagnostic, not an ordinary least-squares reduced chi-square. Values near one are only an approximate goodness-of-fit guide when the model and asymptotic assumptions are appropriate.

Optimization in FLIMage

Differentiating the negative log-likelihood gives

$$\frac{\partial(-\log L)}{\partial \theta_j} = \sum_i \left(1 - \frac{y_i}{\mu_i} \right) \frac{\partial \mu_i}{\partial \theta_j}.$$

FLIMage uses Fisher scoring within its Levenberg-Marquardt solver. For each iteration it forms the working residual

$$e_i = \frac{y_i - \mu_i}{\sqrt{\mu_i}}$$

and the Fisher-weighted model Jacobian

$$J_{ij}^{(F)} = \frac{1}{\sqrt{\mu_i}} \frac{\partial \mu_i}{\partial \theta_j}.$$

The actual Poisson deviance, rather than the working least-squares approximation, determines whether a trial step is accepted.

Before solving the normal equations, FLIMage scales every parameter using the norm of its Jacobian column. This is important because a FLIM fit can combine amplitudes of millions of photons with decay rates near

10^{-2} per bin. Scaling improves numerical conditioning and prevents false convergence, but does not change the likelihood, statistical model, or optimum. It is enabled automatically for all fits that use the shared nonlinear solver.

Poisson ML versus weighted least squares

Poisson-weighted least squares minimizes an approximation such as

$$\sum_i \frac{(y_i - \mu_i)^2}{\sigma_i^2}.$$

If the variance is estimated from the observed histogram, $\sigma_i^2 \approx y_i$, the random observation appears in both the residual and its weight. This can bias low-count fits and requires special handling of bins where $y_i = 0$.

Poisson ML instead uses the probability of the observed integer counts directly. It naturally handles zero-count bins and is generally the preferred objective for raw photon-count histograms. At high counts, the Poisson distribution approaches a Gaussian with variance μ_i , so Poisson ML and properly weighted least squares become increasingly similar.

Unweighted least squares assumes constant variance across bins. That assumption is normally inappropriate for photon-count histograms because Poisson variance equals the expected count.

Practical interpretation

- Poisson ML is the default objective for FLIM decay fitting in the main analysis and time-course paths.
- Include the full model prediction in μ_i , especially the background when it is non-negligible.
- Choose the fit range deliberately. Counts outside the selected range make no contribution to the likelihood.
- A successful convergence code establishes numerical convergence, not that every parameter is identifiable.
- Biexponential fits can have strongly correlated amplitudes and lifetimes, especially at low photon counts or when the lifetimes are similar.
- Compare fitted curves and residuals, test sensitivity to initialization and bounds, and use simulated or profile-likelihood analysis when component parameters are scientifically important.

The systematic behavior of single- and biexponential methods across photon counts and backgrounds is summarized in [Systematic biexponential lifetime benchmark](#).

Math-library usage

For direct use of the fitting library, configure the model and parameter bounds, then select Poisson ML before calling `Per-form()`:

```
var fit = new Fitting.Nlinfit(beta0, x, photonCounts);  
fit.modelFunc = (beta, time) => MyDecayModel(beta, time);  
fit.PoissonMaximumLikelihood();  
  
int result = fit.Perform();  
double[] fittedParameters = fit.beta;  
double reducedDeviance = fit.xi_square;
```

Application code normally selects Poisson ML automatically; users do not need to calculate Poisson weights themselves.